



43. Flask 팀 협업 가이드 (GitHub Fork 방식)

취업 준비생을 위한 실전 협업 프로세스 안내서입니다.

팀장(리더)의 초기 설정과 팀원의 작업·반영 단계로 구성되어 있습니다.

전체 흐름 요약

[팀장] GitHub 조직 생성 → 저장소 생성 → 초기 코드 푸시
[팀원] 저장소 포크 → 로컬 클론 → 브랜치 생성 → 개발 → PR 요청
[팀장] PR 검토 → 머지
[팀원] 최신 코드 동기화 → 반복

시작 전 필수 준비

Git 설치 확인 및 초기 설정

Git을 처음 사용한다면 아래 명령어로 사용자 정보를 먼저 등록해야 합니다.
커밋 기록에 이름과 이메일이 남기 때문에 반드시 먼저 실행해야 합니다.

```
git config --global user.name "홍길동"  
git config --global user.email "your@email.com"
```

설정이 잘 됐는지 확인하려면:

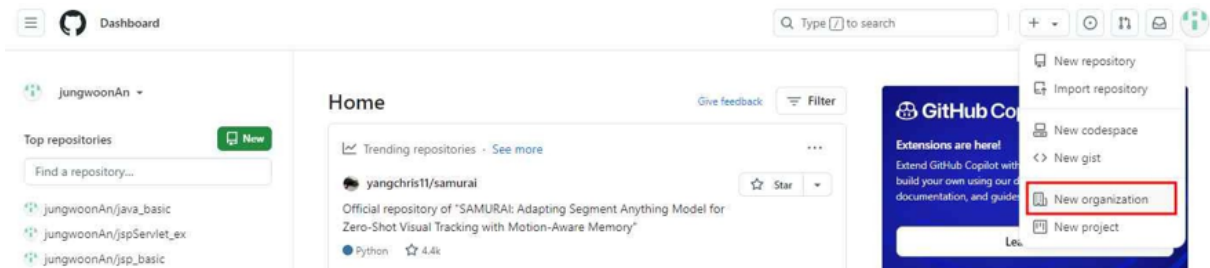
```
git config --list
```



`user.name` 과 `user.email` 이 출력되면 성공입니다.

[팀장 부문] 1단계: GitHub 조직 및 초기 프로젝트 생성

1-1. GitHub Organization 생성 및 팀원 초대



1. GitHub 메인 페이지 우측 상단의 **+** 아이콘 클릭 → **New organization** 선택
2. **Free** 플랜 선택 후 조직 이름과 담당자 이메일 입력하여 생성
3. 조직 관리 페이지의 **People** 탭에서 팀원의 GitHub ID 또는 이메일로 초대

1-2. 로컬 Flask 프로젝트 초기화

① 프로젝트 폴더 생성 및 이동

```
mkdir flask-team-project
cd flask-team-project
```

② 가상환경 설정 및 Flask 설치

```
python -m venv venv
```

가상환경 활성화:

```
# Windows
./venv/Scripts/activate

# Mac / Linux
source venv/bin/activate
```

💡 가상환경 활성화 확인 방법

터미널 프롬프트 맨 앞에 **(venv)** 가 붙으면 활성화된 상태입니다.

```
(venv) C:\Users\사용자명\flask-team-project>
```

(venv) 가 보이지 않는다면 활성화 명령어를 다시 실행하세요.

```
pip install Flask
pip freeze > requirements.txt
```

③ 기본 app.py 파일 생성

```
# app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, Flask Team Project!"

if __name__ == '__main__':
    app.run(debug=True)
```

④ .gitignore 파일 생성 ★ 중요

GitHub에 올라가면 안 되는 파일을 제외하기 위해 프로젝트 루트에 .gitignore 파일을 만들고 아래 내용을 저장합니다.

```
venv/
__pycache__/
*.pyc
.env
instance/
```

⚠ .gitignore 를 빠뜨리면 용량이 큰 가상환경 폴더(venv/)가 GitHub에 통째로 올라갑니다. 반드시 먼저 만드세요!

1-3. Organization 원격 저장소 생성 및 최초 푸시

① GitHub 저장소 생성

- Organization 페이지 → New repository 클릭
- 저장소 이름 입력 (예: flask-web-project)

- **Public** 으로 생성
- ⚠ README나 `.gitignore` 는 생성하지 않고 빈 저장소로 만들기

② 로컬 Git 저장소 초기화 및 푸시

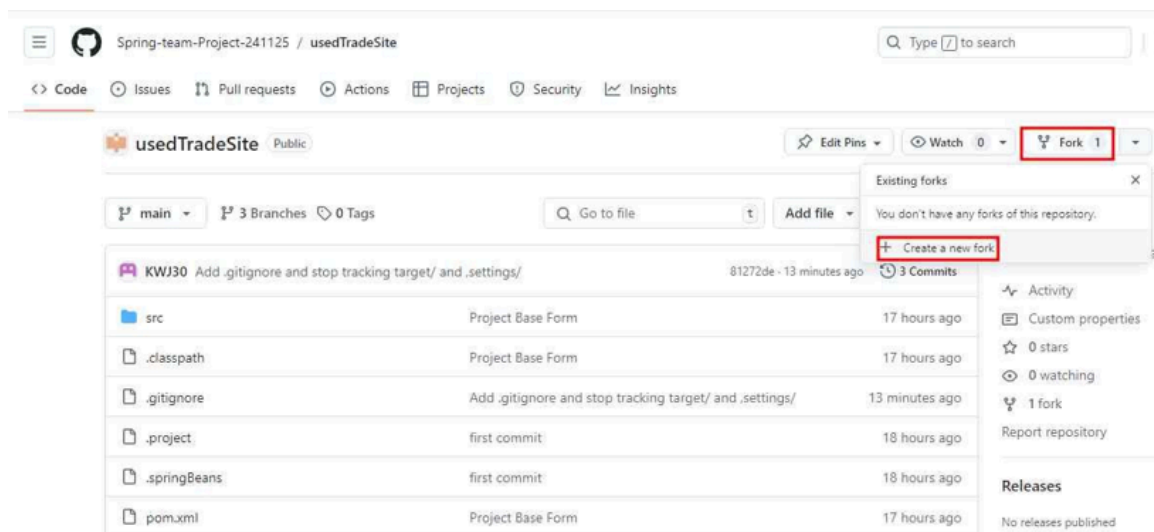
```
git init
git remote add origin https://github.com/조직이름/레포지토리명.git
git branch -M main
git add .
git commit -m "Initialize Flask project"
git push -u origin main
```

💡 위 명령어에서 `조직이름` 과 `레포지토리명` 을 실제 값으로 바꿔 입력하세요.

예: `https://github.com/my-flask-org/flask-web-project.git`

👥 [팀원 부문] 2단계: 저장소 복제 및 개발 준비

2-1. 팀장 저장소 포크(Fork)하기



1. GitHub 로그인 후 팀장이 만든 **조직 레포지토리 주소**로 이동
2. 우측 상단의 **Fork** 버튼 클릭
3. Owner를 **본인의 개인 GitHub 계정**으로 설정 후 **Create fork** 클릭

The screenshot shows the GitHub interface for creating a new fork. At the top, the repository path is 'Spring-team-Project-241125 / usedTradeSite'. Below the navigation bar, the 'Create a new fork' section is visible. It includes a description of forking, a note about required fields, and input fields for 'Owner' (jungwoonAn) and 'Repository name' (usedTradeSite). A checkbox for 'Copy the main branch only' is checked. A green 'Create fork' button is at the bottom right.

포크가 완료되면 **본인계정/레포지토리명** 형태의 복사본 저장소가 생깁니다.

2-2. 내 저장소 클론(Clone) 및 원격 주소 설정

① 로컬에 프로젝트 복제(Clone)

포크한 **본인 계정**의 저장소를 로컬 컴퓨터로 다운로드합니다.

```
git clone https://github.com/본인계정ID/레포지토리명.git
cd 레포지토리명
```

② 가상환경 구성

클론 받은 프로젝트에는 **venv** 폴더가 없으므로, 직접 만들고 패키지를 설치해야 합니다.

```
python -m venv venv

# 가상환경 활성화 (Windows)
./venv/Scripts/activate

# 가상환경 활성화 (Mac / Linux)
source venv/bin/activate

# 패키지 일괄 설치
pip install -r requirements.txt
```

③ 조직(Upstream) 원격 저장소 연결

팀장의 메인 저장소를 `upstream` 이라는 이름으로 등록해, 나중에 최신 코드를 받아올 수 있도록 합니다.

```
git remote add upstream https://github.com/조직이름/레포지토리명.git
```

설정 확인:

```
git remote -v
```

아래와 같이 두 주소가 모두 보이면 성공입니다.

```
origin    https://github.com/본인계정ID/레포지토리명.git (fetch)
origin    https://github.com/본인계정ID/레포지토리명.git (push)
upstream  https://github.com/조직이름/레포지토리명.git (fetch)
upstream  https://github.com/조직이름/레포지토리명.git (push)
```

[팀원 부문] 3단계: 기능 개발 및 코드 올리기

3-1. 브랜치(Branch) 생성 및 작업

 **main** 브랜치에서 직접 코드를 수정하지 마세요!

각자 기능별로 새 브랜치를 만들어 작업하는 것이 팀 협업의 기본 규칙입니다.

① 새로운 기능 브랜치 생성 및 이동

```
# 브랜치 이름 예시: feature/login, feature/signup, feature/main-page
git checkout -b 기능브랜치명
```

② Flask 코드 작성

```
# 가상환경 활성화 (Windows)
./venv/Scripts/activate

# 가상환경 활성화 (Mac / Linux)
source venv/bin/activate
```

```
# Flask 개발 서버 실행 후 코드 작성
python app.py
```

3-2. 커밋 메시지 작성 규칙

커밋 메시지는 **타입: 작업 내용** 형식으로 작성하면 팀원이 변경 이력을 쉽게 파악할 수 있습니다.

타입	의미	예시
feat	새로운 기능 추가	feat: 로그인 기능 구현
fix	버그 수정	fix: 로그인 오류 수정
docs	문서 수정 (README 등)	docs: README 설치 방법 추가
style	코드 포매팅 (기능 변경 없음)	style: 들여쓰기 정리
refactor	코드 리팩토링	refactor: 로그인 함수 분리
chore	기타 잡무 (패키지 설치 등)	chore: requests 패키지 추가

3-3. 본인 원격 저장소(origin)에 푸시하기

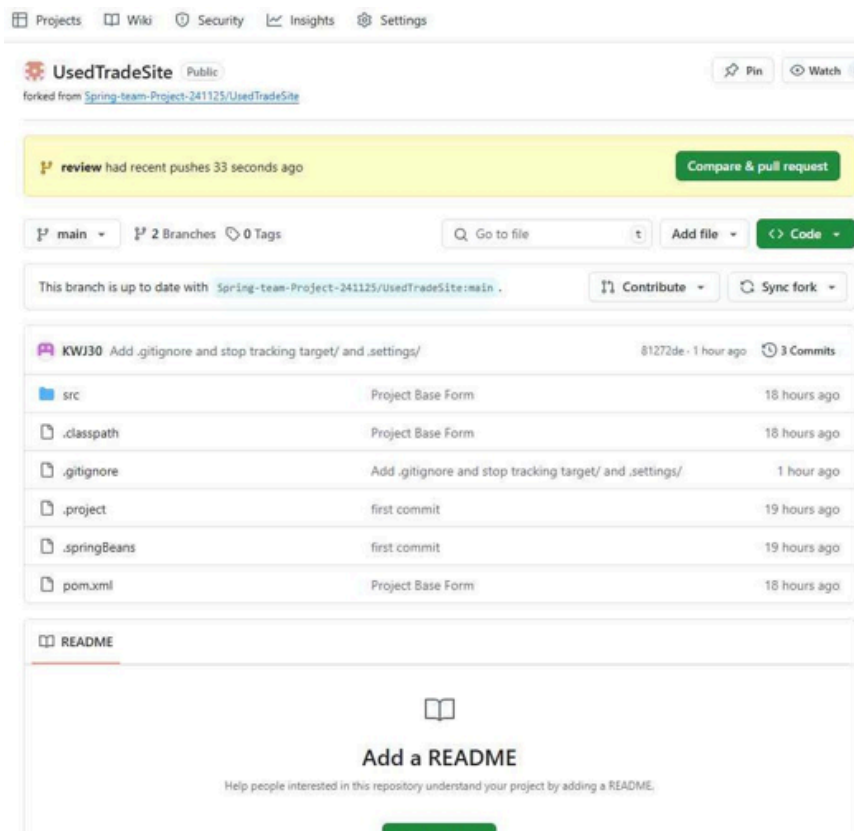
작업이 완료되면 변경 파일을 스테이징하고 커밋한 뒤, **본인의 포크 저장소**로 푸시합니다.

```
git add .
git commit -m "feat: 완료한 기능에 대한 간결한 설명"
git push --set-upstream origin 기능브랜치명
```

[전체 부문] 4단계: 코드 검토(PR) 및 동기화

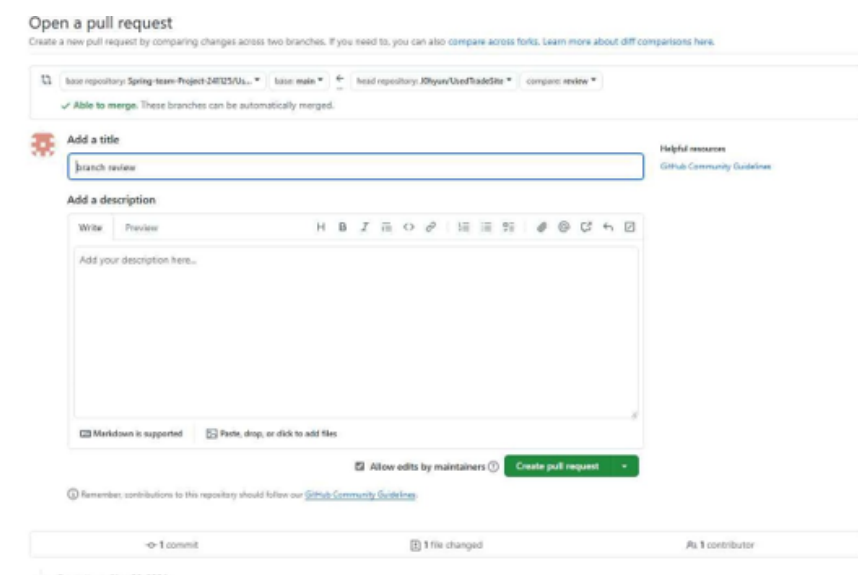
4-1. 팀원: Pull Request(PR) 요청 보내기

1. 본인의 GitHub 포크 저장소 페이지로 이동하면 노란색 바와 함께 **Compare & pull request** 버튼이 나타납니다.

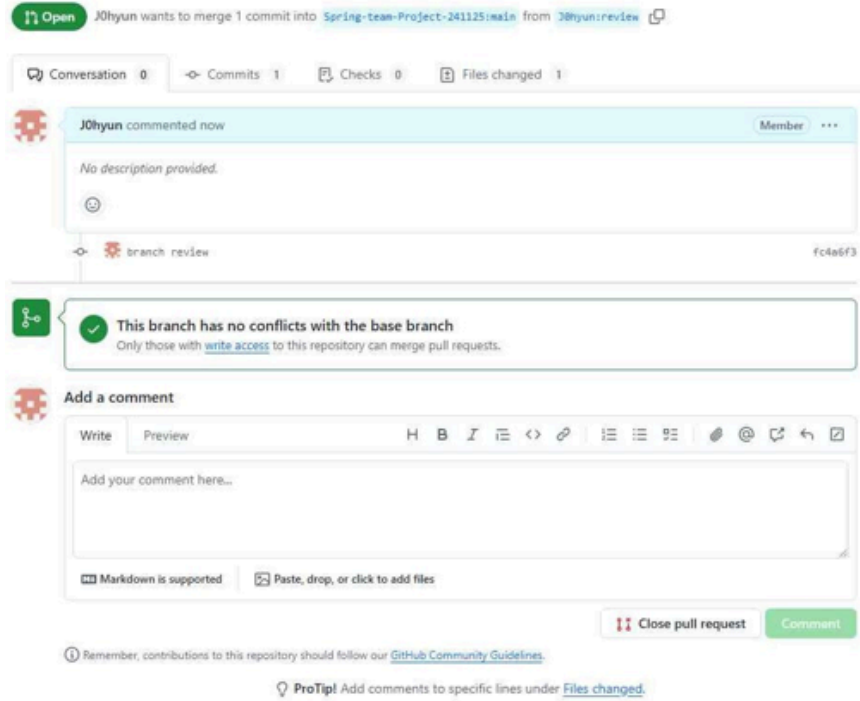


2. **Target(대상) 확인:** 조직 저장소의 **main** 브랜치로 지정되어 있는지 확인합니다.

3. 제목과 작업 내용을 명확하게 작성 후 **Create pull request** 클릭



branch review #2

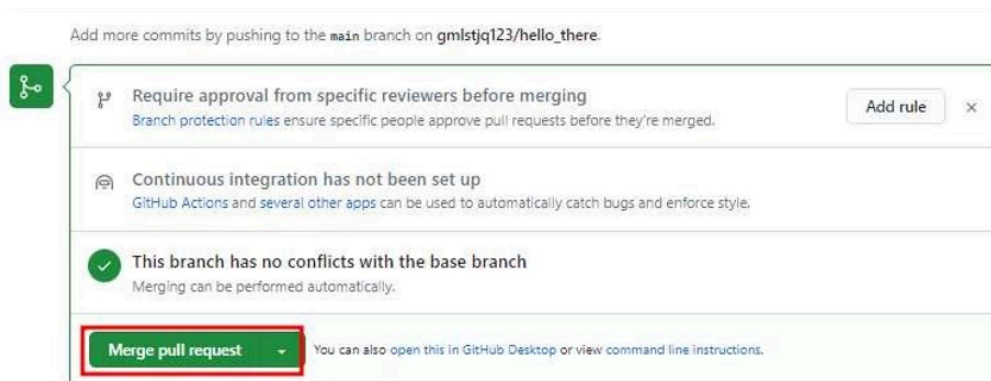


💡 PR 제목 예시: **feat: 로그인 기능 구현 (#이슈번호)**

본문에는 "어떤 기능을 만들었는지", "테스트 방법은 무엇인지" 등을 적어주면 좋습니다.

4-2. 팀장: 코드 검토 및 머지(Merge) 처리

1. 조직 저장소의 **Pull requests** 탭에서 요청 확인
2. 코드 내용을 검토하고 충돌(Conflict)이 없다면 **Merge pull request** → **Confirm merge** 클릭



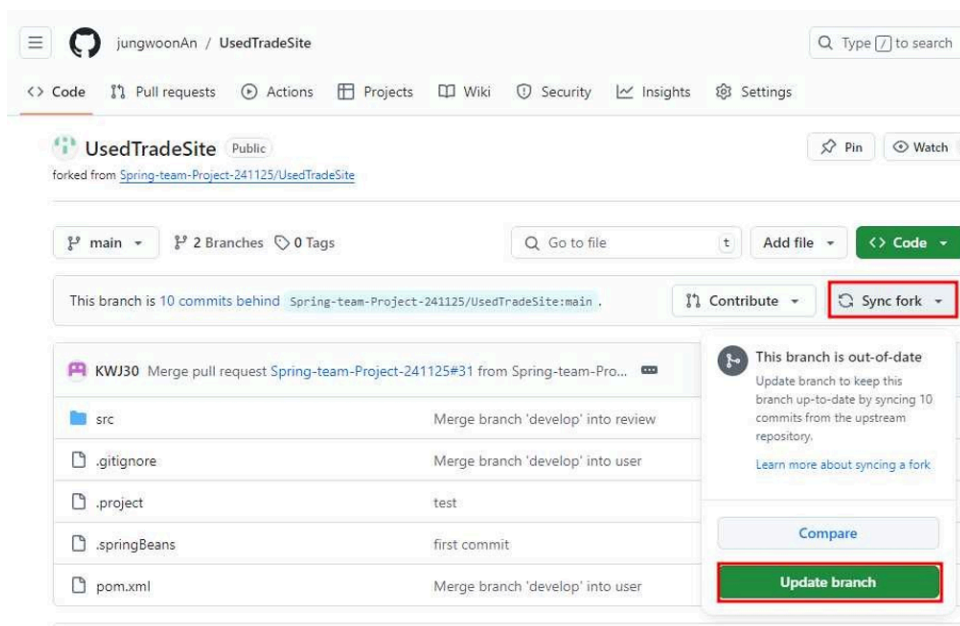


4-3. 팀원: 최신 코드로 동기화하기

다른 팀원의 기능이 메인 저장소에 병합됐다면, 내 로컬과 포크 저장소도 최신 상태로 업데이트해야 충돌을 방지할 수 있습니다.

① GitHub 웹사이트에서 포크 동기화

- 본인의 GitHub 포크 저장소 메인 페이지로 이동
- **Sync fork** 버튼 클릭 → **Update branch** 클릭



② 로컬 컴퓨터로 최신 코드 가져오기

```
git checkout main
git pull origin main
```

③ 내 기능 브랜치에 최신 코드 병합하기

```
git checkout 기능브랜치명
git merge main
```

이후 가상환경을 켜고 다음 기능을 이어서 개발합니다.

⚠ 충돌(Conflict) 발생 시 해결 방법

`git merge main` 실행 후 같은 파일을 여러 명이 수정했다면 아래와 같은 충돌 메시지가 나타납니다.

```
CONFLICT (content): Merge conflict in app.py
Automatic merge failed; fix conflicts and then commit the result.
```

충돌 해결 순서

① 충돌 파일 열기

충돌이 난 파일을 열면 아래와 같은 표시가 있습니다.

```
<<<<<<< HEAD (내 코드)
def home():
    return "나의 변경사항"
=====
def home():
    return "팀원의 변경사항"
>>>>>>> main
```

② 원하는 코드로 직접 수정

`<<<<<<<`, `=====`, `>>>>>>>` 표시를 모두 지우고 최종 코드만 남깁니다.

```
def home():
    return "최종으로 결정한 내용"
```

③ 충돌 해결 후 커밋

```
git add .
git commit -m "fix: app.py 충돌 해결"
```

💡 충돌이 자주 발생한다면 작업 전 항상 `git merge main` 으로 최신 코드를 먼저 반영하는 습관을 들이세요.

🧹 작업 완료 후 브랜치 정리

PR이 머지되면 해당 기능 브랜치는 더 이상 필요하지 않습니다. 브랜치를 정리하는 습관을 들이면 저장소를 깔끔하게 유지할 수 있습니다.

```
# 로컬 브랜치 삭제
git branch -d 기능브랜치명

# 원격(GitHub) 브랜치 삭제
git push origin --delete 기능브랜치명
```

💡 GitHub 웹사이트에서 PR 머지 후 `Delete branch` 버튼을 눌러 원격 브랜치를 바로 삭제할 수도 있습니다.

🔄 전체 반복 사이클 요약

- ① `git checkout main` # main 브랜치로 이동
- ② `git pull origin main` # 최신 코드 가져오기
- ③ `git checkout -b 기능브랜치` # 새 브랜치 생성
- ④ (코드 작성...)
- ⑤ `git add .`
- ⑥ `git commit -m "feat: ..."`
- ⑦ `git push origin 기능브랜치`
- ⑧ GitHub에서 PR 요청
- ⑨ 팀장 머지 후 브랜치 삭제
- ⑩ 처음으로 돌아가 반복

📌 처음에는 복잡해 보이지만, 3번만 반복하면 손에 익습니다.
막히는 부분이 있으면 팀원과 공유하며 함께 해결해 보세요! 💪